PROGRAMAÇÃO DE SOFTWARE BÁSICO EM C

Trabalho Prático Integrador
Relatório técnico

Sistema de Pedidos da Lanchonete

Gerenciamento e monitoramento de tarefas com processos, threads, pipe e arquivos

Giovanna Borneo
Felipe Bezerra

Resumo

Este trabalho apresenta uma aplicação modular de terminal, escrita em C para Linux, que utiliza pedidos de uma lanchonete como representação de tarefas. O sistema cadastra, lista e consulta pedidos, simula o preparo, salva e recupera dados em TXT, cria um processo filho e transmite seu resultado por pipe. Também prepara dois pedidos com threads, protege o histórico com mutex e implementa medições repetidas de consulta.

As escolhas priorizam uma implementação pequena, com vetor fixo de estruturas e funções diretas. O relatório descreve a arquitetura, os fluxos de execução, o tratamento de erros e o plano de testes, distinguindo implementação inspecionada de validação efetivamente realizada.

> **Estado desta versão:** documentação para revisão técnica. A compilação C, os testes de execução e a coleta de resultados no Codespaces ainda estão pendentes. Não são apresentados resultados experimentais fictícios.

Versão documental: 03 de novembro de 2026.

PARTE 1 · INTRODUÇÃO E OBJETIVOS

1. Problema, proposta e escopo

O enunciado do Trabalho Prático Integrador propõe uma aplicação local de gerenciamento e monitoramento de tarefas. Além de armazenar registros, a solução deve executar operações, criar processos, utilizar threads, estabelecer comunicação entre processos e registrar resultados. O objetivo didático é integrar conteúdos da linguagem C às APIs de um sistema operacional [1].

Para tornar essas operações mais fáceis de demonstrar, foi adotado o tema de uma lanchonete. Uma tarefa corresponde a um pedido; executar a tarefa significa simular seu preparo; criar um processo corresponde a abrir uma estação de preparo. O processo principal representa o caixa, que recebe o resultado da estação por um pipe.

1.1 Objetivo geral

Desenvolver uma aplicação modular em C que permita cadastrar, executar e acompanhar pedidos, integrando persistência em arquivos, processos, threads, comunicação local e observação do tempo de operações.

1.2 Objetivos específicos

- Representar pedidos com estruturas, strings, constantes e um vetor de capacidade limitada.
- Oferecer menu numerado com cadastro, listagem, consulta e validação de entradas.
- Persistir e recuperar dados em TXT sem biblioteca externa de serialização.
- Criar um filho com `fork()`, aguardar seu término e comunicar o resultado por pipe.
- Executar dois pedidos com `pthreads`, avaliando os dados compartilhados e o uso de mutex.
- Registrar data/hora, PID, pedido, estado e duração; repetir medições e calcular estatísticas.

1.3 Delimitação

O preparo é uma simulação com `sleep()` de um ou dois segundos. Ele não representa um equipamento real nem um experimento de processamento intensivo. O sistema atende uma sessão local de terminal e não implementa rede, banco de dados, interface gráfica ou vários caixas sincronizados.

Salvar e carregar são ações manuais. O programa não promete recuperação transacional, backup automático ou coordenação entre instâncias independentes. Comparações de velocidade entre execução sequencial e concorrente não fazem parte do escopo atual.

> O tema altera os nomes da situação-problema, mas preserva os conceitos solicitados. A adequação final da simulação à avaliação depende da apreciação do professor; este relatório não presume uma aprovação individual já registrada.

PARTE 2 · FUNDAMENTAÇÃO TEÓRICA E REQUISITOS

2. Conceitos utilizados

2.1 Estruturas, vetores, strings e ponteiros

O tipo `Pedido`, declarado em `include/pedido.h`, agrupa ID, descrição, tempo de preparo e estado. A descrição usa um vetor de 100 posições de `char`, das quais uma é reservada ao terminador NUL. O vetor principal comporta até 50 pedidos; apenas as posições anteriores ao contador `total` são consideradas ocupadas.

O cadastro recebe `total` por ponteiro para atualizar o contador mantido em `main()`. A busca retorna o índice do pedido ou `-1`. O preparo recebe o endereço de um pedido e altera seu estado diretamente. Essas escolhas permitem demonstrar passagem de argumentos, endereçamento e acesso a estruturas sem introduzir alocação dinâmica.

2.2 Processo e comunicação

Um processo possui seu espaço de memória. Após `fork()`, o filho começa com uma cópia do estado do pai, mas uma mudança no pedido do filho não atualiza o vetor do pai. Por isso, a estação envia o estado final por um pipe anônimo, e o processo principal aplica o resultado recebido. `waitpid()` permite aguardar e verificar o término do filho [2–4].

2.3 Threads e sincronização

Threads de um mesmo processo compartilham memória. O programa passa endereços de pedidos diferentes a duas threads e aguarda ambas com `pthread_join()`. Como cada thread modifica um registro distinto e o menu aguarda os joins, esse fluxo não exige mutex para os campos dos pedidos. Já o arquivo de histórico é usado pelas duas threads, portanto sua gravação é protegida [5–7].

2.4 Arquivos e relógios

A persistência utiliza a interface `FILE *` da biblioteca C. Abertura, escrita, leitura e fechamento são verificados. Os registros dos pedidos são separados por ponto e vírgula; o histórico utiliza append. A escolha de TXT favorece a inspeção do conteúdo no editor.

Data/hora de calendário identifica quando um log foi gerado. Para durações, o programa utiliza `CLOCK_MONOTONIC`: esse relógio é adequado a intervalos e não deve ser confundido com o tempo de CPU que seria medido por `clock()` [8].

2.5 Ambiente

A aplicação tem como alvo Linux/POSIX, compilação com GCC e `-Wall -Wextra -pthread`. O Codespaces possui configuração de ambiente baseada em Ubuntu 24.04. Python 3 é ferramenta de teste, não dependência de execução do programa C. Não se pretende compatibilidade direta com o terminal nativo do Windows.

PARTE 2 · MAPEAMENTO DOS REQUISITOS

3. Requisitos e acesso pelo menu

A tabela relaciona os requisitos obrigatórios às operações implementadas. O estado indicado é de **implementação por inspeção**; a compilação e a comprovação em execução continuam pendentes nesta versão.

RF	Operação e localização	Menu
01	Cadastrar ID, descrição e tempo; recusar duplicidade e vetor cheio. `src/pedido.c`	1
02	Listar os registros cadastrados e seus estados. `src/pedido.c`	2
03	Buscar pelo identificador e exibir o pedido. `src/pedido.c` e `src/main.c`	3
04	Simular o preparo e atualizar para PRONTO. `src/pedido.c`	4, 5, 6
05	Salvar o vetor em `dados/pedidos.txt`· `src/arquivo.c`	9
06	Carregar e validar em vetor temporário. Há limitação conhecida com NUL. `src/arquivo.c`	10
07	Criar processo filho com `fork()` e aguardar com `waitpid()`· `src/processo.c`	5
08	Transmitir o estado do filho para o pai por pipe anônimo. `src/processo.c`	5
09	Criar duas threads e aguardar seus joins. `src/thread.c`	6
10	Registrar data/hora, PID, pedido, estado e duração; consultar o histórico. `src/log.c`	4–7
11	Medir cinco consultas, menor, maior e média; analisar limites. `src/main.c`	8
12	Tratar entradas, arquivos, relógio, processos, threads e IPC nos módulos correspondentes.	Transversal

Regras de entrada

O menu usa leitura de linha completa e `strtol()`. Prefixos como `1abc`, números fora da faixa de `int`, linhas vazias e linhas excessivas são recusados. A linha inválida é consumida para não deixar restos para a próxima leitura. EOF normal encerra; erro real de leitura encerra com falha.

O ID pode ser zero ou negativo, desde que único. A descrição aceita até 99 bytes e pode ser vazia, mas não aceita ponto e vírgula, NUL ou CR interno na entrada interativa. O cadastro restringe o preparo a um ou dois segundos. Esses limites são decisões do projeto, não requisitos adicionais atribuídos ao professor.

PARTE 3 · ARQUITETURA E DESENVOLVIMENTO

4. Organização e fluxo básico

O código foi organizado em pastas simples. Os arquivos `.c` ficam em `src/`; os cabeçalhos em `include/`. As suítes permanecem em `tests/`, os dados de execução em `dados/` e a documentação em `docs/`. O executável e o Makefile permanecem na raiz, que também é o diretório de execução.

Módulo	Responsabilidade principal
`main.c`	Menu, leitura, decisões de interface e medição das consultas.
`pedido.c`	Cadastro, listagem, busca por ID e preparo de um pedido.
`arquivo.c`	Persistência e recuperação dos pedidos.
`processo.c`	Filho, descritores do pipe, envio/recebimento e espera.
`thread.c`	Validação do par, criação e espera das threads.
`log.c`	Histórico, data/hora e mutex da gravação.

4.1 Estrutura e preparo sequencial

`main()` mantém o vetor `Pedido pedidos[MAX_PEDIDOS]` e o contador. As operações recebem esses dados por parâmetro. O vetor fixo dispensa gerenciamento de memória dinâmica, ao custo de limitar a quantidade de registros e utilizar busca linear. Para o tamanho escolhido, essa solução é direta e fácil de explicar.

A opção 4 encontra o ID e chama `prepararPedido()`. A função verifica o ponteiro, o estado e o tempo, inicia a medição, executa a espera e atualiza para PRONTO. Se um sinal interromper `sleep()`, a espera é retomada com o tempo restante. O término gera tentativa de registro no histórico.

Um pedido pronto não é preparado novamente. Se o relógio inicial falhar, o preparo não começa. Se a medição final ou o log falharem depois do preparo, o estado permanece PRONTO, com aviso. Essa decisão evita afirmar que uma operação já concluída foi desfeita.

4.2 Compilação e reorganização

```
make
./programa
make clean
```

O Makefile compila os seis fontes numa única regra, usando `-Iinclude -Wall -Wextra -pthread`. Os cinco cabeçalhos são dependências do executável. A reorganização preservou os 11 arquivos C/cabeçalhos byte a byte, verificados por SHA-256; apenas caminhos do build e dos testes foram adaptados. Isso comprova preservação de conteúdo, não funcionamento do executável.

PARTE 4 · PROCESSOS, THREADS E IPC

5. Estação de preparo e pipe

A opção 5 recebe um ID existente e chama `abrirEstacaoPreparo()`. O pedido deve estar pendente e possuir tempo permitido. O menu aguarda a estação terminar: não há execução em segundo plano.

5.1 Fluxo da comunicação

1. O pai descarrega a saída com `fflush(stdout)`, evitando repetição do menu que poderia estar no buffer herdado.
2. `pipe(canais)` cria a ponta de leitura `canais[0]` e a de escrita `canais[1]`. O pipe existe antes do `fork()`.
3. O filho fecha a leitura, informa seu PID e chama `prepararPedido()`. A medição e o log ocorrem no filho.
4. O filho envia um inteiro com o estado final, fecha a escrita, descarrega suas mensagens e termina com `_exit()`, sem retornar ao menu.
5. O pai fecha a escrita, recebe o inteiro, fecha a leitura e aguarda aquele filho com `waitpid()`.
6. O pai só aplica PRONTO se recebeu a mensagem completa, o filho terminou normalmente com código zero e o estado recebido é válido.

5.2 Dados transmitidos e sincronização

A mensagem é o estado final de um único pedido. O ID e o tempo já são conhecidos pelo pai; por isso não são reenviados. Um inteiro nativo é suficiente para o pipe entre pai e filho do mesmo programa. Esse formato não é um protocolo de rede portável entre máquinas diferentes.

Receber o estado resolve a separação de memória: a alteração feita pelo filho é aplicada explicitamente ao pedido do pai. A espera evita continuar o menu antes de verificar o encerramento. As pontas não utilizadas são fechadas para liberar recursos e permitir a detecção de fim de arquivo no canal.

5.3 Tratamento de falhas e limites

São verificados `pipe()`, `fork()`, `read()`, `write()`, `close()` e `waitpid()`. Leituras e escritas parciais são completadas; interrupções `EINTR` de E/S e espera são retomadas. Após criar o filho, o pai tenta recolhê-lo mesmo se a comunicação falhar.

Se o filho preparar e registrar o pedido, mas o resultado não for confirmado pelo pai, o log pode existir enquanto o vetor principal continua pendente. Não há transação entre preparo, log, pipe e TXT. Uma falha que feche a leitura pode causar `SIGPIPE` no filho; o pai não confirma um término anormal.

> Na demonstração devem ser observados PIDs distintos, mensagem completa pelo pipe e consulta do estado no pai. Nesta versão, esse roteiro está preparado, mas sua execução C ainda não foi comprovada.

PARTE 4 · CONCORRÊNCIA E SINCRONIZAÇÃO

6. Dois pedidos com threads

A opção 6 recebe exatamente dois IDs diferentes. `prepararPedidosParalelo()` verifica os pedidos antes de iniciar qualquer thread: ambos precisam existir, estar pendentes e ter tempo de um ou dois segundos. Pedidos iguais ou inválidos são recusados.

6.1 Criação e compartilhamento

O módulo realiza as duas chamadas de `pthread_create()` antes de começar os `pthread_join()`. Se um join fosse colocado imediatamente após cada criação, o primeiro preparo terminaria antes do segundo começar, eliminando a concorrência pretendida.

A função da thread recebe um `void *` que aponta para o pedido correspondente e reutiliza `prepararPedido()`. Não se cria uma cópia do vetor como acontece no processo filho. Ao concluir, cada thread já alterou o registro da memória compartilhada, e não é necessário transmitir seu estado por pipe.

As duas threads escrevem em elementos distintos. O menu não consulta nem salva durante o preparo, pois aguarda os joins. Portanto, o fluxo atual não utiliza mutex em torno do vetor. Essa justificativa depende de manter essas condições; adicionar acesso simultâneo aos mesmos pedidos exigiria nova análise.

6.2 Histórico protegido

O histórico é o recurso compartilhado pelas duas threads. Em `registrarLog()`, um mutex estático protege a abertura em append, a escrita da linha e o fechamento do arquivo. O buffer é descarregado antes de liberar a trava. Data/hora são formatadas com `localtime_r()`, utilizando armazenamento próprio de cada chamada.

O mutex não envolve `sleep()`. Assim, a espera dos preparos pode se sobrepor; apenas o acesso ao arquivo é serializado. A ordem dos logs não é determinada. Os registros possuem o mesmo PID porque as duas threads pertencem ao mesmo processo.

6.3 Falhas e resultados parciais

Os erros de pthreads são apresentados a partir do código retornado pela função, e não de `errno`. Se a segunda criação falhar, a primeira thread ainda é aguardada e pode deixar um pedido pronto. Um preparo bem-sucedido não é desfeito por falha no outro.

Se um join falhar, o módulo tenta os demais e indica término não confirmado; o programa encerra com erro sem retomar o menu. Uma falha comum de arquivo libera o mutex e mantém o pedido pronto. Falha excepcional ao liberar a trava também provoca encerramento, evitando continuar com um estado incerto.

> O mutex coordena somente threads desse processo. Não protege várias instâncias independentes do programa nem garante recuperação de escrita parcial. Não se afirma ganho de velocidade antes de medir execuções reais.

PARTE 5 · PERSISTÊNCIA, LOGS E TESTES

7. Arquivos de pedidos e histórico

7.1 Escolha do formato TXT

O enunciado permite texto ou binário, com justificativa [1, p. 12]. Foi escolhido TXT para permitir inspeção no editor, facilitar a demonstração e usar apenas a biblioteca C. Cada linha contém quatro campos: ID, descrição, tempo em segundos e estado numérico.

```
1;X-burger e suco;2;0
2;Batata frita;1;1
```

Exemplo ilustrativo: não representa saída coletada em um teste. Estado 0 = PENDENTE; 1 = PRONTO.

`salvarPedidos()` abre `dados/pedidos.txt` para escrita e substitui o conteúdo anterior. Lista vazia gera arquivo vazio. O retorno de abertura, de cada escrita e do fechamento é verificado; uma falha pode aparecer somente ao descarregar o buffer no fechamento.

7.2 Recuperação e preservação do estado

`carregarPedidos()` lê para um vetor temporário. Se o conteúdo passar pela validação, substitui a lista e restaura os estados. Formato inválido, campos numéricos fora do limite, estados desconhecidos, IDs repetidos e excesso de registros são recusados. Um arquivo vazio substitui a lista por zero pedidos.

O carregamento preserva valores legados de tempo, mas o preparo recusa tempos fora de um ou dois segundos. Salvar e carregar são sempre manuais; sair não salva, e abrir uma nova sessão não carrega automaticamente.

Há uma limitação conhecida: `fgets()` seguido de funções de string pode interpretar apenas o prefixo anterior a um NUL embutido. Assim, um sufixo inválido pode ficar invisível ao parser. A rejeição desse caso e seu teste de regressão ainda precisam ser implementados. O salvamento também não mantém backup e não é atômico.

7.3 Registro das execuções

As opções de preparo acrescentam registros em `dados/historico.txt`. A opção 7 consulta sem criar o arquivo e informa quando não há histórico. Cadastro, salvamento e carregamento não geram eventos de execução.

```
03/11/2026 14:30:00 | PID=5231 | PEDIDO=3 |
STATUS=PRONTO | TEMPO=2.001 s
```

Exemplo ilustrativo, dividido em duas linhas apenas para apresentação; o programa grava uma entrada por linha.

Data/hora e PID são reais no momento da gravação; a duração vem do preparo. Valores negativos, não finitos ou estados desconhecidos são recusados. Os TXT gerados são ignorados pelo Git. O programa deve ser executado na raiz com `dados/` existente; a limpeza do executável não remove esses dados.

PARTE 5 · COBERTURA E ESTADO DA VALIDAÇÃO

8. Estratégia de testes

Há sete suítes Python com 99 métodos: persistência (13), logs (11), preparo (12), processos (12), threads (18), desempenho (12) e entradas (21). Cenários parametrizados com `subTest` podem exercitar mais de uma condição por método. A quantidade de métodos não equivale a testes aprovados.

As suítes compilam aplicação e drivers em diretórios temporários, com `-Wall -Wextra -Werror -pthread`. Cada cenário executa em outra pasta temporária com seus próprios dados. Wrappers do linker GNU simulam falhas apenas nos testes; não entram no programa principal. Os 15 comandos de compilação foram adaptados à nova organização.

8.1 Dez cenários mínimos do enunciado

Caso	Resultado esperado	Cobertura preparada
1. Cadastro válido	Registro criado.	Entradas e persistência.
2. ID duplicado no cadastro	Recusa, preservando o original.	Teste específico ausente.
3. Consulta válida	Dados exibidos.	Persistência e entradas.
4. Consulta inexistente	Mensagem de não encontrado.	Persistência.
5. Salvar	TXT atualizado.	Persistência.
6. Carregar	Dados recuperados.	Persistência.
7. Criar processo	Filho e PIDs distintos.	Processos.
8. IPC	Resultado recebido no pai.	Processos.
9. Threads	Dois pedidos executados.	Threads.
10. Entrada inválida	Erro tratado e recuperação.	Entradas.

Duplicidade no arquivo é testada, mas não substitui o cenário de repetir o ID pelo cadastro. Também falta testar NUL embutido no TXT. Essas lacunas de cobertura não significam que a recusa de ID duplicado esteja ausente no código.

8.2 Resultado efetivamente obtido

A sintaxe Python dos sete arquivos foi validada. No ambiente local de 03/11/2026, não há GCC nem make, inclusive no Ubuntu/WSL verificado. A descoberta produziu:

```
Ran 0 tests
OK (skipped=7)
```

Foram puladas sete classes; nenhum teste C foi executado. O próximo passo é compilar no Codespaces e executar `python3 -B -m unittest discover -s tests -v`, registrando aprovações, falhas e skips, além das capturas exigidas. Não há nesta versão evidências finais de funcionamento.

PARTE 6 · DESEMPENHO, PROBLEMAS E CONCLUSÃO

9. Método de medição

A operação selecionada foi a consulta de pedido por ID, que corresponde ao RF03 e é utilizada pelo menu. A função `medirDesempenho()`, em `src/main.c`, mede cinco chamadas de `consultarPedido()`. Antes delas, uma consulta preliminar confirma que o ID existe; essa verificação não faz parte dos cinco intervalos.

9.1 Coleta e estatísticas

Para cada repetição, o programa lê `CLOCK_MONOTONIC`, executa a consulta e lê o relógio novamente. Entrada, impressão e cálculo das estatísticas ficam fora do intervalo. O resultado da busca é verificado. Falhas do relógio, intervalo negativo ou resultado inesperado cancelam o experimento sem resumo parcial.

```
tempo_ms = (fim.tv_sec - inicio.tv_sec) * 1000.0
         + (fim.tv_nsec - inicio.tv_nsec) / 1000000.0
```

Após coletar os cinco tempos, o programa apresenta cada execução, o menor, o maior e a média aritmética. Os valores usam milissegundos com seis casas decimais. O experimento não prepara pedidos, não altera estados, não salva o TXT e não gera logs de preparo.

Indicador previsto	Definição	Resultado real
Cinco execuções	Uma busca medida por repetição.	A coletar.
Menor	Mínimo dos cinco tempos.	A coletar.
Maior	Máximo dos cinco tempos.	A coletar.
Média	Soma dos cinco tempos dividida por cinco.	A coletar.

9.2 Limitações e interpretação

Cinco repetições atendem ao mínimo do enunciado, mas não demonstram estabilidade estatística. Uma busca linear em até 50 registros é muito curta. Resolução e custo do relógio, chamadas de função, cache e carga do sistema podem dominar o intervalo. Tempos iguais a zero são aceitos; a quantidade de casas exibidas não garante essa precisão.

A posição do ID no vetor também influencia a busca. Repetir o mesmo ID não representa todos os casos possíveis. Na coleta final, devem ser anotados o ambiente, a quantidade de pedidos, o ID e as condições de execução, evitando conclusões amplas com uma amostra pequena.

> Não foram coletados tempos reais nesta revisão. Os testes com relógio controlado verificam cálculos quando executados, mas não substituem um experimento real. Não se afirma aceleração por threads nem comparação sequencial versus concorrente.

PARTE 6 · LIMITAÇÕES E FECHAMENTO

10. Problemas, decisões e conclusão

10.1 Decisões de implementação

O vetor fixo, os módulos pequenos e o TXT foram escolhidos para reduzir dependências e facilitar a explicação. O custo é uma capacidade máxima de 50 registros, busca linear e restrição do separador na descrição. A simulação curta torna o fluxo de execução observável, mas não representa uma carga real de processamento.

Os caminhos relativos simplificam o programa, porém exigem execução pela raiz. A reorganização em `src/`, `include/` e `docs/` melhora a visualização sem mudar a lógica dos módulos. O Makefile e os testes foram ajustados para essa separação; o build real ainda precisa ser confirmado.

10.2 Limitações conhecidas

- **Validação incompleta:** falta compilação C, execução das suítes e evidências reais no ambiente de destino.
- **Leitura do TXT:** tratamento de NUL embutido pendente; um prefixo válido pode ocultar conteúdo inválido após esse byte.
- **Cobertura:** falta um teste específico de duplicidade no cadastro, embora a checagem exista na função de cadastro.
- **Persistência manual:** encerrar sem salvar perde alterações em memória. Não há backup nem substituição atômica do TXT.
- **Falhas parciais:** preparo concluído pode ficar sem log; falha de comunicação pode deixar o pai sem confirmação do preparo do filho.
- **Concorrência local:** mutex não coordena vários programas. O menu aguarda processos e threads, sem monitoramento em segundo plano.
- **Desempenho:** cinco consultas curtas não sustentam conclusão de ganho de velocidade ou estabilidade estatística.

10.3 Próximos passos

A prioridade é compilar e executar no Codespaces, corrigir e testar o caso NUL e completar o cenário de ID duplicado. Depois, registrar os dez casos mínimos, coletar as medições e atualizar este relatório com resultados observados. Slides e capturas finais ainda precisam ser produzidos. O vídeo preservado em `docs/evidencias/fase1.mp4` documenta apenas o menu inicial.

10.4 Conclusão

Por inspeção, o projeto reúne os elementos solicitados: fundamentos de C, armazenamento, execução simulada, processo filho, pipe, threads, mutex, logs e estatísticas. A separação entre memória de processos e memória compartilhada de threads aparece em dois fluxos distintos e explicáveis.

A entrega funcional, entretanto, não pode ser considerada comprovada apenas pela existência do código ou de testes preparados. O fechamento depende da validação real e da compreensão dos módulos por ambos os autores, que deverão demonstrar o sistema ao vivo na defesa de 10–15 minutos.

REFERÊNCIAS E LOCALIZAÇÃO DOS ARTEFATOS

Referências

[1] **Trabalho Prático Integrador — Programação de Software Básico em C: Sistema de Gerenciamento e Monitoramento de Tarefas.** Enunciado fornecido pelo professor, 25 páginas. Referências principais: requisitos (p. 7), persistência (p. 12), logs (p. 13), medição (p. 14), organização (p. 16), testes (p. 18), relatório (p. 20), entregáveis (p. 22) e checklist (p. 25). Arquivo local de apoio: `data/Trabalho-Pratico-Integrador.pdf`; a pasta é ignorada pelo Git.

[2] Linux man-pages. **fork(2).**
https://man7.org/linux/man-pages/man2/fork.2.html

[3] Linux man-pages. **pipe(2).**
https://man7.org/linux/man-pages/man2/pipe.2.html

[4] Linux man-pages. **wait(2): wait(), waitpid() e waitid().**
https://man7.org/linux/man-pages/man2/wait.2.html

[5] Linux man-pages. **pthread_create(3).**
https://man7.org/linux/man-pages/man3/pthread_create.3.html

[6] Linux man-pages. **pthread_join(3).**
https://man7.org/linux/man-pages/man3/pthread_join.3.html

[7] The Open Group. **pthread_mutex_lock(), pthread_mutex_trylock(), pthread_mutex_unlock().** POSIX.1-2017.
https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_mutex_lock.html

[8] Linux man-pages. **clock_gettime(2).**
https://man7.org/linux/man-pages/man2/clock_gettime.2.html

Artefatos do projeto

O `README.md` contém instruções, menu, estrutura e pendências. Os módulos estão em `src/` e `include/`; as suítes estão em `tests/`. A fonte editável deste documento é `docs/relatorio.html`, e sua versão de leitura é `docs/relatorio.pdf`. O código C não depende de ferramentas de geração de PDF.

Roteiro de fechamento da entrega

1. Compilar com os avisos exigidos e executar as sete suítes, sem confundir skips com aprovação.
2. Registrar os dez casos mínimos, logs, PIDs, IPC, threads e medições reais.
3. Atualizar relatório, preparar slides e ensaiar a demonstração ao vivo.

Autoria: Giovanna Borneo e Felipe Bezerra. Documento sem matrículas, instituição ou identificação docente, pois esses dados não foram informados para esta versão.